

da supera un cierto valor permitido), expandimos *la siguiente página*, es decir, la página $p - 2^t$ (¡que no es necesariamente la que produjo el rebalse que llevó a exceder el costo promedio de búsqueda permitido!). Esta página se lee y sus elementos y se reinsertan en las páginas $h(y) \bmod 2^{t+1}$, es decir, se reparten entre la misma $p - 2^t$ y la nueva página p , que se agrega al final del archivo. Al terminar la expansión, hacemos $p \leftarrow p + 1$, y si resulta que $p = 2^{t+1}$, entonces hemos completado una expansión y nos preparamos para la siguiente: $t \leftarrow t + 1$. Para eliminar un valor, éste se busca en la tabla y simplemente se elimina. En caso de estar en una lista de rebalse, puede usarse su espacio para mover un elemento desde la última página de la lista, de modo de poder liberar apenas se pueda esta última página y reducir así el tiempo promedio de búsqueda. Si se eliminan suficientes elementos, puede resultar que la tabla pueda *contraerse* sin que se exceda el costo promedio de búsqueda máximo permitido. Para realizar una contracción, primero se hace $t \leftarrow t - 1$ si $p = 2^t$, y luego se hace $p \leftarrow p - 1$. Luego, se agregan todos los elementos de la página p a la página $p - 2^t$, procediendo a eliminar la página p . Note que esto puede hacer rebalsar la página $p - 2^t$, o alargar su lista de rebalse.

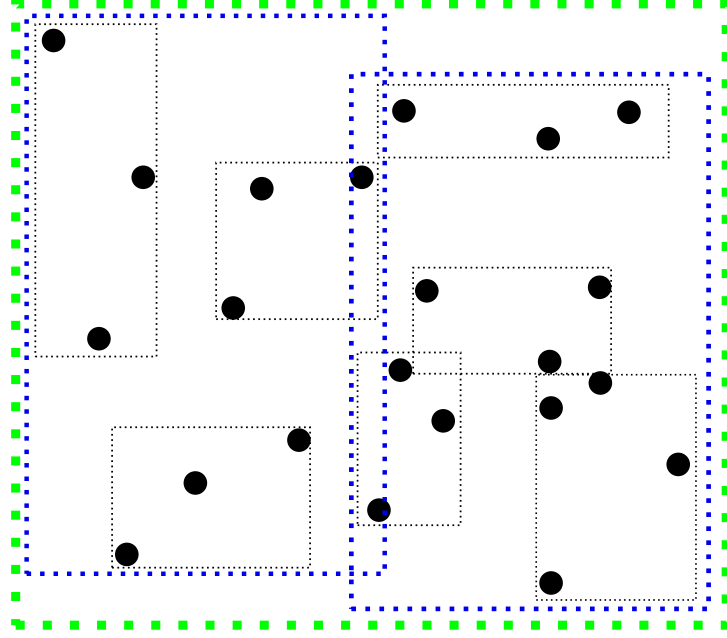
Note que una expansión o una contracción no necesariamente cambiarán la condición que las disparó acerca del costo promedio de búsqueda. En general es preferible, para evitar que una inserción o borrado disparen muchas expansiones o contracciones, realizar de todas maneras una sola, y dejar que la siguiente operación dispare nuevamente una expansión o contracción, hasta que la situación se resuelva.

Se pueden usar otros criterios en vez del máximo costo promedio de búsqueda. Por ejemplo, puede permitirse un mínimo porcentaje de llenado de las páginas, de modo de contraer cuando éste se hace demasiado bajo (y expandir cuando es posible sin violar el criterio). Este criterio se contrapone al de mantener un costo de búsqueda máximo, por lo que sólo se puede controlar uno de los dos (o puede usarse una combinación de criterios). En general el hashing lineal se comporta mejor que el extendible, aunque no suele garantizar un solo acceso por lectura.

2.6. R-trees

El R-tree es la estructura de datos más popular para almacenar puntos o hiperrectángulos, en 2 o más dimensiones. Es una extensión de la idea del B-tree, en el sentido de que los nodos usan bloques de disco garantizando una fracción mínima de llenado, tiene altura $O(\log_B N)$, las hojas están todas al mismo nivel, y usa mecanismos similares de inserción y borrado.

Las “claves” son *minimum bounding boxes (MBBs)*, es decir, el menor hiperrectángulo que encierra un conjunto de objetos. El R-tree puede representar objetos complejos, y la



clave de búsqueda es su MBB (como caso particular, podemos también almacenar puntos). Cada nodo interno tiene k claves y k hijos. La clave y_i que almacena para su hijo T_i es el MBB de los MBBs almacenados en la raíz de T_i . Dado un parámetro $0 < \alpha \leq \frac{1}{2}$, el R-tree garantiza que todo nodo u hoja, excepto la raíz, tiene entre αB y B claves.

El R-tree permite encontrar todos los objetos cuyos MBBs se intersecten con un hiperrectángulo de consulta. La forma de proceder es leer la raíz, comparar la consulta con los k MBBs que almacena, y recursivamente continuar la búsqueda en todos los subárboles T_i tal que y_i se intersecta con la consulta. Cuando se llega a las hojas, los MBBs intersectados se reportan. Asimismo, se puede usar para encontrar todos los objetos del R-tree que contienen al de la consulta, mediante entrar en todos los hijos cuyos MBBs contengan a la consulta.

Note que la consulta puede entrar a cero o más hijos de un nodo, por lo que el tiempo de búsqueda no es $O(\log_B N)$. Podemos obtener una complejidad promedio si consideramos una probabilidad fija p de que la consulta se intersecte con un MBB. Entonces el tiempo promedio cumple la recurrencia $T(N) = 1 + pB \cdot T(N/B)$, cuya solución es $O(N^{1-\log_B(1/p)})$. Es decir, la complejidad es de la forma $O(n^\beta)$ para un $0 < \beta < 1$ que depende de la probabilidad de intersección. Es por ello que es importante lograr que los MBBs sean lo más pequeños posible, mediante una adecuada política de inserción y borrado de objetos.

Para insertar un objeto x , partimos de la raíz y vemos si está completamente contenido en algún MBB. Si lo está, elegimos el de menor área y continuamos. Si no, calculamos en qué hijo T_i tendríamos que incrementar menos el área de la clave y_i al convertirla en $y'_i = MBB(y_i \cup x)$, reemplazamos y_i por y'_i y continuamos la inserción en T_i . Finalmente, al llegar a una hoja, agregamos x a los objetos.

En caso de que la hoja pase a tener $B + 1$ objetos, debemos partirla en dos hojas de modo de minimizar la suma de las áreas de ambos MBBs y que ninguna tenga menos de αB objetos. Note que en el modelo de memoria externa podríamos considerar las $\Theta(2^B)$

particiones posibles y seleccionar la mejor, lo que sería “gratis” en términos de I/O, pero esto es impracticable en la realidad por su costo de CPU. En cambio, se utilizan heurísticas. Una clásica, llamada “quadratic split”, hace lo siguiente:

- Escoge dos claves y e y' lo más alejadas posible, es decir, que maximicen las áreas de $MBB(y \cup y') - MBB(y) - MBB(y')$. Esto toma tiempo $O(B^2)$ en memoria principal. Estas claves serán los primeros elementos de las dos nuevas hojas.
- Va insertando las demás claves, eligiendo en cada paso la que incremente menos el área al insertarla en el MBB de la hoja de y o en el de la hoja de y' . En caso de empate, puede escoger la hoja de menor área o la de menos elementos. Esto también cuesta $O(B^2)$ operaciones en memoria principal.
- Cuando una hoja llegue a $(1 - \alpha)B$ elementos, los demás van a la otra.

Este mecanismo se usa también cuando los nodos internos rebalsan. En total, una inserción cuesta $O(\log_B N)$ I/Os, y $O(B^2 \log_B N)$ tiempo de CPU.

Para borrar un elemento, se elimina de su hoja y se calcula su nuevo MBB, que puede decrecer. A la vuelta de la recursión, se pueden ir recalculando los MBBs de los ancestros del nodo, al costo de una nueva escritura del bloque. Cuando una hoja tiene menos de αB elementos, en vez de intentar algo parecido al B-tree, es decir, unirla con una hoja vecina y de ser necesario volverlas a partir, lo que hace el R-tree es eliminarla completamente y reinsertar todos los elementos en el árbol. Esto suele mejorar el empaquetamiento de objetos en MBBs. Note que esto puede ocurrir también con subárboles completos, cuando un nodo interno queda con menos de αB claves.

Almacenando los primeros $\Theta(\log_B M)$ niveles en memoria principal, el costo de inserción se reduce a $O(\log_B \frac{n}{m})$ I/Os.

2.7. Ficha Resumen

- Árbol B: $O(\log_B \frac{n}{m})$ para insertar, borrar y buscar. Óptimo para buscar si se procede por comparaciones. Ocupación promedio 69 %. Permite recuperar todos los *occ* objetos en un rango en tiempo óptimo $O(\log_B \frac{n}{m} + occ/B)$, y encontrar el predecesor y sucesor de un elemento en tiempo $O(\log_B \frac{n}{m})$.
- Ordenamiento: $O(n \log_m n)$ (o, equivalentemente, $O(n \log_m \frac{n}{m})$), lo que es óptimo si se procede por comparaciones.
- Cola de prioridad: $O(\frac{1}{B} \log_m n)$ (amortizado) si $\log \frac{n}{m} \leq m^\alpha$ para una constante $0 < \alpha < 1$, lo que es óptimo si se procede por comparaciones.
- Hashing extendible: para $N = O(MB)$. Inserta, borra y busca en $O(1)$ promedio, con un buen hash. Con uno que produce t colisiones, el costo es $O(\lceil t/B \rceil)$. Ocupación promedio 69 %.